이 pdf 자료 다 읽고 난뒤 이 코드 실행하여 보세요.
.2.코드_허깅페이스 klue/bert-base 로 긍부정분류- 데이터셋활용법포함 .ipynb
- NSMC데이터셋다운로드(영화 리뷰 텍스트, 긍/부정 레이블)
- csv자료를 허깅페이스 데이터셋으로 변환하여 파인튜닝
- 데이터셋함수로 만들어진 객체 자료 관련 코드
- 코드 마지막에 LLM 특화 평가: 프롬프트 강건성 + Few-shot 테스트

1. 데이터셋 형태의 변환 중요성: 판다스프레임->허깅페이스데이터셋

AI 모델을 개발할 때, 데이터의 역할은 매우 중요합니다. 하지만 데이터는 다양한 형태와 구조로 존재하며, 이를 그대로 모델에 적용하기는 어렵습니다. 모델이 데이터를 학습하고 이해하려면, 데이터의 형식을 모델에 맞게 변환해야 합니다.
허깅페이스의 경우, 데이터셋을 표준화된 Dataset 또는 DatasetDict 객체로 변환하면 다양한 이점을 얻을 수 있습니다.

▶주요 이점

허깅페이스 객체로 변환하면, 복잡한 데이터 관리 코드를 작성할 필요 없이, 허깅페이스 생태계의 모든 도구들을 효과적으로 활용할 수 있습니다.

모델 호환성	대부분의 허깅페이스 모델은 Dataset 객체를 입력으로 받도록 설계되었습니다. 따라서 데이터를 이 형식으로 변환하면, 별도의 추가 작업 없이 다양한 모델에 손쉽게 적용할 수 있습니다.
성능 최적화	허깅페이스 데이터셋은 대규모 데이터 처리에 최적화되어 있습니다. 메모리에 한 번에 올릴 수 없는 대용량 데이터도 효율적으로 처리하고, 멀티프로세싱을 활용하여 데이터 로딩 속도를 향상시킵니다.
편의성	데이터셋 객체는 map, filter, shuffle과 같은 유용한 메서드를 제공합니다. 이를 통해 전처리, 데이터 필터링, 섞기 등 다양한 작업을 간단하게 수행할 수 있습니다. 마치 엑셀에서 데이터를 쉽게 정렬하고 필터링하는 것처럼, 코드를 더 간결하고 효율적으로 작성할 수 있습니다.

2. 허깅페이스 from datasets 모듈을 이용한 파인튜닝에 사용할 데이터변환

1) 단일 Dataset 객체-> from datasets import Dataset

A. 딕셔너리자료-> 허깅페이스의 데이터셋으로 변환

```
from datasets import Dataset
data = {
    'text': ['이것은 첫 번째 문장입니다.', '두 번째 문장입니다.'],
    'label': [0, 1]
}
dataset = Dataset.from_dict(data)
```

```
# 출력 결과
# Dataset({
#     features: ['text', 'label'],
#     num_rows: 2
# })
```

B. List자료 -> 허깅페이스의 데이터셋으로 변환

```
from datasets import Dataset
# 리스트 형태로 데이터 준비
data_list = [
    {'text': '첫 번째 문장', 'label': 1},
    {'text': '두 번째 문장', 'label': 0}
]
# Dataset 객체로 변환
my_dataset_from_list = Dataset.from_list(data_list)
print(my_dataset_from_list)
```

```
# 출력 결과
# Dataset({
#     features: ['text', 'label'],
#     num_rows: 2
# })
```

C. 판다스자료 -> 허깅페이스의 데이터셋으로 변환

```
data = {
    'text': ['오늘은 날씨가 정말 좋네요.', '정말 지루한 영화였어요.',
             '이 음식은 맛이 없네요.'],
    'label': [1, 0, 0]
}
df = pd.DataFrame(data)

print("Pandas DataFrame:")
hf_dataset = Dataset.from_pandas(df)
print("\n\nHugging Face Dataset:")
print(hf_dataset)
```

```
Dataset({
    features: ['text', 'label'],
    num_rows: 3
})
```

2) DatasetDict 객체 -> from datasets import DatasetDict

DatasetDict 객체는 학습(train), 검증(validation), 테스트(test) 데이터셋을 한 번에 관리하기 위해 사용하는 딕셔너리 형태의 객체입니다.
단일 Dataset 객체가 하나의 데이터셋을 담는다면, DatasetDict는 여러 개의 Dataset 객체를 키-값 쌍으로 묶어 놓은 것입니다.

▶ DatasetDict의 주요 특징

구조	파이썬 딕셔너리처럼 동작하며, 키('train', 'validation', 'test')는 데이터셋의 분할(split)을 나타내고, 값은 각각의 Dataset 객체가 됩니다.
활용성	머신러닝 모델을 학습시킬 때, 일반적으로 학습용, 검증용, 테스트용 데이터를 나누어 사용합니다. DatasetDict를 사용하면 이 세 가지 데이터셋을 하나의 변수에서 통합적으로 관리하고 쉽게 접근할 수 있습니다.
편의성	허깅페이스의 Trainer API와 같은 여러 도구들은 DatasetDict를 입력으로 받도록 설계되어 있어, 별도의 데이터 로딩 코드 없이 효율적으로 학습 파이프라인을 구축할 수 있습니다.

```
from datasets import Dataset, DatasetDict

# 1. 각 데이터셋의 데이터를 딕셔너리로 정의
train_data_dict = {'text': ['학습용 텍스트 1', '학습용 텍스트 2']}
val_data_dict = {'text': ['검증용 텍스트 1']}
test_data_dict = {'text': ['테스트용 텍스트 1']}

# 2. Dataset 객체로 변환
train_dataset = Dataset.from_dict(train_data_dict)
val_dataset = Dataset.from_dict(val_data_dict)
test_dataset = Dataset.from_dict(test_data_dict)
```

```
DatasetDict({
    train: Dataset({
        features: ['text'],
        num_rows: 2
    }),
    validation: Dataset({
        features: ['text'],
        num_rows: 1
    }),
    test: Dataset({
        features: ['text'],
        num_rows: 1
```

```
# 3. DatasetDict에 통합할 딕셔너리 구성
datasets_to_load = {
    'train': train_dataset,
    'validation': val_dataset,
    'test': test_dataset
}

# 4. DatasetDict 객체 생성
my_dataset_dict = DatasetDict(datasets_to_load)

# 결과 확인
print(my_dataset_dict)
print(my_dataset_dict)
```

```
    })
})
```

- csv자료 -> 허깅페이스 DataDict로 변환 가능함

```
from datasets import load_dataset

# 로컬 CSV 파일로부터 데이터셋 로드
# train.csv, validation.csv, test.csv 파일이 있다고 가정
# load_dataset은 여러 파일을 불러와 DatasetDict로 구성 가능

my_dataset_dict = load_dataset("csv", data_files={
    "train": "train.csv",
    "validation": "validation.csv",
    "test": "test.csv"
})

print(my_dataset_dict)
```

```
# 출력 결과
# DatasetDict({
#     train: Dataset(...),
#     validation: Dataset(...),
#     test: Dataset(...)
# })
```

3. Dataset 주요함수

A. map함수

함수는 데이터셋의 모든 항목에 동일한 함수를 적용하는 데 사용됩니다. 즉, 데이터셋의 각 행(row)에 대해 지정된 전처리 함수를 실행하고, 그 결과를 새로운 데이터셋으로 반환합니다
이 함수는 다음과 같은 상황에서 매우 유용합니다.

토크나이징(Tokenizing)	텍스트 데이터를 모델이 이해할 수 있는 토큰 ID로 변환합니다.
특징(Feature) 추가/수정	새로운 컬럼을 추가하거나, 기존 컬럼의 값을 변경합니다
필요 없는 컬럼 삭제	학습에 불필요한 컬럼을 제거합니다.

예1: 텍스트에 접두사 추가하기

```
from datasets import Dataset

# 1. 예제 데이터셋 생성
raw_dataset = Dataset.from_dict({'text': ['안녕하세요', '반갑습니다']})
print("원본 데이터셋:")
print(raw_dataset)
```

```
원본 데이터셋:
Dataset({
    features: ['text'],
    num_rows: 2
})
전처리된 데이터셋:
Dataset({
```

# 2. 전처리 함수 정의 def add_prefix(example): example['text'] = "접두사: " + example['text'] return example # 3. map 함수로 전처리 적용 processed_dataset = raw_dataset.map(add_prefix) print("\n전처리된 데이터셋:") print(processed_dataset)	features: ['text'], num_rows: 2 })
print(processed_dataset[0]) print(list(processed_dataset['text'])) print(processed_dataset['text'])	{'text': '접두사: 안녕하세요'} ['접두사: 안녕하세요', '접두사: 반갑습니다'] Column(['접두사: 안녕하세요', '접두사: 반갑습니다'])

B. map함수 옵션

옵션	기능	중요성
batched=True	배치 단위 처리	가장 중요. 전처리 속도와 효율성 극대화.
remove_columns	불필요한 컬럼 삭제	메모리 절약 및 데이터셋 간결화.
with_indices	인덱스 정보 전달	특정 샘플에 대한 조건부 처리나 디버깅에 활용.
num_proc	멀티프로세싱 활성화	CPU 병렬 처리로 전처리 시간 단축.

(1) batched=True

- **기능**:전처리 함수가 한 번에 하나의 샘플이 아닌, 여러 개의 샘플(배치)을 처리하도록 합니다.
- **중요성**:

성능 최적화	토크나이저 같은 대부분의 전처리 함수는 배치 단위로 처리할 때 병렬 연산을 활용해 훨씬 빠르게 작동합니다. 이 옵션은 특히 대규모 데이터셋을 다룰 때 전처리 속도를 획기적으로 단축시킵니다.
효율성	여러 샘플을 한 번에 처리하므로 메모리와 연산 효율성이 높아집니다.

- **사용 예시**: 텍스트를 토크나이징할 때 batched=True를 사용하면 한 번의 함수 호출로 여러 문장을 동시에 변환할 수 있습니다, 하지만 batched=False로 하면 하나씩만 처리되어 느림

(2) remove_columns

- **기능**: 전처리 후 더 이상 필요 없는 원본 컬럼을 데이터셋에서 제거합니다.
- **중요성**:

메모리 절약	모델 학습에는 보통 input_ids나 attention_mask 같은 토큰화된 데이터만 필요합니다. 원본 텍스트 컬럼을 제거하면 불필요한 메모리 낭비를 줄일 수 있습니다.
깔끔한 데이터셋 관리	최종적으로 모델에 입력될 데이터만 남겨놓아 데이터셋의 구조를 간결하고 명확하게 유지합니다

(3) with_indices

- **기능**: 전처리 함수에 각 샘플의 인덱스(index)를 전달합니다.
- **중요성**: 데이터셋의 인덱스 정보를 활용해 특정 조건에 따라 다른 전처리를 적용하거나, 디버깅을 위해 특정 샘플을 추적할 때 유용합니다.

(4) num_proc

- **기능**: 전전처리 작업을 처리할 프로세스 수를 지정합니다.
- **중요성**:

멀티프로세싱	여러 CPU 코어를 활용해 전처리 작업을 병렬로 처리함으로써, 특히 batched=False와 함께 사용할 때나 전처리 함수가 CPU 연산을 많이 요구할 때 전체 작업 시간을 단축할 수 있습니다.
성능 조정	시스템의 CPU 코어 수에 맞춰 num_proc 값을 설정하면 전처리 효율성을 극대화할 수 있습니다.

(참조) map함수에 대한 Code 참조

```
# datasets map 함수의 batched와 remove_columns 학습
from datasets import Dataset
from transformers import AutoTokenizer
# 샘플 데이터 준비
data = {
    'text': ['안녕하세요', '좋은 하루입니다', '감사합니다', '안녕히 가세요'],
    'label': [0, 1, 1, 0],
    'extra_info': ['정보1', '정보2', '정보3', '정보4']
}
dataset = Dataset.from_dict(data)
tokenizer = AutoTokenizer.from_pretrained("klue/bert-base")

print("원본 데이터셋:")
print(dataset)
print(f"컬럼: {dataset.column_names}")
```

```
[출력물]
원본 데이터셋:
Dataset({
    features: ['text', 'label', 'extra_info'],
    num_rows: 4
})
컬럼: ['text', 'label', 'extra_info']
```

```
# ================================================================================
# 1. batched=False (기본값) - 하나씩 처리
# ================================================================================
print("\n" + "="*60)
print("1. batched=False (하나씩 처리)")
print("="*60)

def tokenize_single(example):
    """하나의 샘플씩 처리"""
    print(f"처리 중: {example['text']}")  # 처리 과정 확인
    return tokenizer(example['text'], truncation=True, max_length=10)

# batched=False로 처리
result_single = dataset.map(tokenize_single, batched=False)
print(f"\n결과 컬럼: {result_single.column_names}")
print("첫 번째 결과:")
print(result_single[0])
```

```
Map: 100%
 4/4 [00:00<00:00, 44.92 examples/s]
처리 중: 안녕하세요
처리 중: 좋은 하루입니다
처리 중: 감사합니다
처리 중: 안녕히 가세요

결과 컬럼: ['text', 'label', 'extra_info', 'input_ids', 'token_type_ids', 'attention_mask']
첫 번째 결과:
{'text': '안녕하세요', 'label': 0, 'extra_info': '정보1',
'input_ids': [2, 5891, 2205, 5971, 3],
'token_type_ids': [0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1]}
```

```
# ================================================================================
# 2. batched=True - 배치로 처리
# ================================================================================
print("\n" + "="*60)
print("2. batched=True (배치로 처리)")
print("="*60)

def tokenize_batch(examples):
    """여러 샘플을 배치로 처리"""
    print(f"배치 크기: {len(examples['text'])}")
    print(f"배치 내용: {examples['text']}")
    return tokenizer(examples['text'], truncation=True, max_length=10)

# batched=True로 처리 (기본 배치 크기: 1000)
result_batch = dataset.map(tokenize_batch, batched=True)
print(f"\n결과 컬럼: {result_batch.column_names}")
print("첫 번째 결과:")
print(result_batch[0])
```

```
Map: 100%
 4/4 [00:00<00:00, 89.73 examples/s]
배치 크기: 4
배치 내용: ['안녕하세요', '좋은 하루입니다', '감사합니다', '안녕히 가세요']

결과 컬럼: ['text', 'label', 'extra_info', 'input_ids', 'token_type_ids', 'attention_mask']
첫 번째 결과:
{'text': '안녕하세요', 'label': 0, 'extra_info': '정보1', 'input_ids': [2, 5891, 2205, 5971, 3],
'token_type_ids': [0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1]}
```

```
# ================================================================================
# 3. batch_size 조정
# ================================================================================
print("\n" + "="*60)
print("3. batch_size 조정")
print("="*60)

# 작은 배치 크기로 처리
result_small_batch = dataset.map(
    tokenize_batch,
    batched=True,
    batch_size=2  # 2개씩 처리
)
print("batch_size=2로 처리 완료")
```

```
Map: 100%
 4/4 [00:00<00:00, 51.52 examples/s]
배치 크기: 2
배치 내용: ['안녕하세요', '좋은 하루입니다']
배치 크기: 2
배치 내용: ['감사합니다', '안녕히 가세요']
batch_size=2로 처리 완료
```

```
# ================================================================================
# 4. remove_columns 없이 처리
# ================================================================================
print("\n" + "="*60)
print("4. remove_columns 없이 처리")
print("="*60)

def simple_tokenize(examples):
    return tokenizer(examples['text'], truncation=True, max_length=10)

# remove_columns 없음 - 원본 컬럼들이 그대로 남음
result_no_remove = dataset.map(simple_tokenize, batched=True)
print(f"결과 컬럼 (remove 없음): {result_no_remove.column_names}")
print("첫 번째 결과:")
for key, value in result_no_remove[0].items():
    if key == 'input_ids':
        print(f"  {key}: {value}")
    else:
        print(f"  {key}: {value}")
```

```
Map: 100%
 4/4 [00:00<00:00, 64.84 examples/s]
결과 컬럼 (remove 없음): ['text', 'label', 'extra_info', 'input_ids', 'token_type_ids', 'attention_mask']
첫 번째 결과:
  text: 안녕하세요
  label: 0
  extra_info: 정보1
  input_ids: [2, 5891, 2205, 5971, 3]
  token_type_ids: [0, 0, 0, 0, 0]
  attention_mask: [1, 1, 1, 1, 1]
```

```
# ================================================================================
# 5. remove_columns 사용
# ================================================================================
print("\n" + "="*60)
print("5. remove_columns 사용")
print("="*60)

# 특정 컬럼들 제거
result_remove_some = dataset.map(
    simple_tokenize,
    batched=True,
    remove_columns=['text']  # 'text' 컬럼만 제거
)
print(f"결과 컬럼 (text 제거): {result_remove_some.column_names}")

# 여러 컬럼 제거
result_remove_multi = dataset.map(
    simple_tokenize,
    batched=True,
    remove_columns=['text', 'extra_info']  # 여러 컬럼 제거
)
print(f"결과 컬럼 (text, extra_info 제거): {result_remove_multi.column_names}")

# 모든 원본 컬럼 제거
result_remove_all = dataset.map(
    simple_tokenize,
    batched=True,
    remove_columns=dataset.column_names  # 모든 원본 컬럼 제거
)
print(f"결과 컬럼 (모든 원본 제거): {result_remove_all.column_names}")
```

```
Map: 100%
 4/4 [00:00<00:00, 54.31 examples/s]
결과 컬럼 (text 제거): ['label', 'extra_info', 'input_ids', 'token_type_ids', 'attention_mask']
Map: 100%
 4/4 [00:00<00:00, 70.86 examples/s]
결과 컬럼 (text, extra_info 제거): ['label', 'input_ids', 'token_type_ids', 'attention_mask']
Map: 100%
 4/4 [00:00<00:00, 71.68 examples/s]
결과 컬럼 (모든 원본 제거): ['input_ids', 'token_type_ids', 'attention_mask']
```

*.2.코드_허깅페이스 klue/bert-base 로 긍부정분류- 데이터셋활용법포함 .ipynb
- NSMC데이터셋다운로드(영화 리뷰 텍스트, 긍/부정 레이블)
- csv자료를 허깅페이스 데이터셋으로 변환하여 파인튜닝
- 데이터셋함수로 만들어진 객체 자료 관련 코드
- 코드 마지막에 LLM 특화 평가: 프롬프트 강건성 + Few-shot 테스트